

Analisi e Progettazione del Software

Appunti di Analisi e Progettazione del Software

Anno Accademico: 2024/25

Mattia Cosimi

*Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche
Università degli Studi “Roma Tre”*

Indice

1	Pattern GRASP	1
1.1	Introduzione	1
1.2	Creator (Creatore)	1
1.3	Information Expert (Esperto delle informazioni)	1
1.4	Low Coupling (Accoppiamento Basso)	2
1.5	Controller (Controllore)	2
1.6	High Cohesion (Coesione Alta)	2
1.7	Pure Fabrication	3
1.8	Polymorphism	3
2	Pattern GoF	3
2.1	Strategy	3
3	Appunti per Monopoly Game	4
3.1	Iterazione 2	4
3.2	Iterazione 3	4

1 Pattern GRASP

1.1 Introduzione

Nella progettazione OO, un **pattern** è una descrizione, con un nome, di un problema di progettazione ricorrente e di una sua soluzione ben provata e che può essere applicata a nuovi contesti.

1.2 Creator (Creatore)

Uno dei problemi comuni che occorre affrontare nella progettazione a oggetti è: chi crea l'oggetto X? Si tratta di una responsabilità di **fare**. L'obiettivo è quello di avere un **salto rappresentazionale basso** tra il modo in cui si pensa al dominio e una corrispondenza diretta con gli oggetti software.

Problema: chi crea l'oggetto A?

Soluzione: Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è):

- B "contiene" o aggrega con una composizione oggetti di tipo A.
- B registra A
- B utilizza strettamente A.
- B possiede i dati per l'inizializzazione di A.

B e A fanno riferimento a oggetti software, non a oggetti del modello di dominio. Si ricordi che una pratica della modellazione agile consiste nel creare, in parallelo, modelli complementari, dinamici e statici degli oggetti. Pertanto si disegna sia un diagramma di sequenza che un diagramma delle classi, entrambi parziali. NOTA: in monopolyGame il creator è Board.

1.3 Information Expert (Esperto delle informazioni)

Il pattern Information Expert è uno dei principi di base per l'assegnazione delle responsabilità nella progettazione a oggetti. Nell'esempio di MonopolyGame: "Si supponga che alcuni oggetti debbano essere in grado di trovare una particolare Square, dato il suo nome univoco". Si tratta in questo caso di una responsabilità di **conoscere**, ma Expert si applica anche a responsabilità di fare.

Problema: Qual'è un principio di base per assegnare responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarla.

Una responsabilità necessita di informazioni per essere soddisfatta: informazioni su altri oggetti, sullo stato di un oggetto, sul mondo che circonda l'oggetto, informazioni che l'oggetto può ricavare e così via. NOTA: in monopolyGame l'expert è Board.

1.4 Low Coupling (Accoppiamento Basso)

L'**accoppiamento** è una misura di quanto fortemente un elemento è connesso ad altri elementi, li conosce o dipende da essi. Se esiste un accoppiamento o una dipendenza, quando l'elemento da cui si dipende subisce una modifica, ciò può ripercuotersi sull'elemento dipendente.

Problema: Come ridurre l'impatto dei cambiamenti?

Soluzione: Assegna le responsabilità in modo tale che l'accoppiamento (non necessario) rimanga basso. Usa questo principio per valutare le alternative.

Low Coupling è utilizzato per valutare progetti esistenti o per valutare la scelta tra nuove alternative. Va preferito un progetto in cui l'accoppiamento è più basso che nelle alternative. Un'accoppiamento basso, tende a ridurre il tempo, i costi e i difetti nella modifica del software. Expert chiede di trovare l'oggetto che possiede la maggior parte delle informazioni richieste per la responsabilità e di assegnare ad esso la responsabilità. Se si mette la responsabilità altrove, l'accoppiamento complessivo sarà più alto.

1.5 Controller (Controllore)

Un'architettura a strati semplice ha, tra gli altri, uno strato per l'interfaccia utente e uno strato del dominio.

Problema: Qual'è il primo oggetto oltre lo strato UI a ricevere e coordinare ("controllare") un'operazione di sistema?

Soluzione: Assegna la responsabilità a un oggetto che rappresenta una delle seguenti scelte:

- Rappresenta il "sistema" complessivo, un "oggetto radice", un dispositivo all'interno del quale viene eseguito il software, un punto d'accesso al software o un sottosistema principale (sono tutte varianti di un **facade controller**). (utilizzeremo solo questo)
- Rappresenta uno scenario di un caso d'uso all'interno del quale si verifica l'operazione di sistema (un controller di caso d'uso o controller di sessione).

1.6 High Cohesion (Coesione Alta)

Nella progettazione del software, una qualità fondamentale nota come **coesione** è una misura di quanto sono correlate le operazioni di un elemento software da un punto di vista funzionale, e altresì una misura di quanto lavoro sta eseguendo un elemento software. Sono indicatori della coesione di un oggetto sia la quantità di codice che la correlazione del codice. Coesione cattiva e accoppiamento cattivo vanno spesso di pari passo.

Problema: Come mantenere gli oggetti focalizzati, comprensibili e gestibili e, come effetto collaterale, sostenere Low Coupling?

Soluzione: Assegna le responsabilità in modo tale che la coesione rimanga alta. Usa questo principio per valutare le alternative.

1.7 Pure Fabrication

Problema: Quale oggetto deve avere la responsabilità quando non si vogliono violare High Cohesion e Low Coupling, o altri obiettivi, ma le soluzioni suggerite da Expert (per esempio) non sono appropriate?

Soluzione: Assegna un insieme di responsabilità altamente coeso a una classe artificiale o di convenienza, che non rappresenta un concetto del dominio del problema, ma piuttosto è una classe "inventata", per sostenere coesione alta, accoppiamento basso e riuso. (Da utilizzare quando siamo disperati)(Se la maggior parte dei dati all'interno degli oggetti viene passata ad altri oggetti che devono ragionare su tali dati probabilmente non serve).

1.8 Polymorphism

Problema: Come gestire alternative basate sul tipo? Come creare componenti software inseribili? (1. Eliminare i molti if then else o case o switch che renderebbero difficoltosa la modifica. 2. considerando i componenti in una relazione client server com'è possibile sostituire un componente server con un altro senza ripercussioni sui client?.)

Soluzione: Quando le alternative o i comportamenti correlati variano con il tipo (la classe), allora assegna la responsabilità del comportamento ai tipi per i quali il comportamento varia, utilizzando operazioni polimorfe. (Nella superclasse indicare *abstract* sul metodo polimorfo).

Motivazioni valide per partizionare una classe in sottoclassi: La sottoclasse ha proprietà strutturali aggiuntive di interesse (come attributi e associazioni). La sottoclasse è caratterizzata da un comportamento diverso.

2 Pattern GoF

2.1 Strategy

Problema: Come progettare per gestire un insieme di algoritmi o politiche variabili ma correlati? Come progettare per consentire di modificare questi algoritmi o politiche?

Soluzione: definisci ciascun algoritmo/politica/strategia in una classe separata, con un'interfaccia comune. L'oggetto contesto è l'oggetto a cui va applicata la strategia e delega parte del lavoro al suo oggetto strategia. Un oggetto strategia (Strategy) è invece un oggetto che implementa una strategia, ed è quindi in grado di applicarla solitamente ha lo stesso nome dell'operazione dell'oggetto contesto che supporta. è comune che l'oggetto strategia usi/interroghi l'oggetto contesto.

3 Appunti per Monopoly Game

3.1 Iterazione 2

Nell'interazione due cambiano alcuni comportamenti giustificando la scelta di utilizzare una classe polimorfa. Riguardo al lancio dei dadi ci sono dei problemi, inizialmente è il giocatore a tirare i dadi e sommarne il totale, creando dei problemi, come ad esempio: non si può semplicemente chiedere il totale dei dadi senza doverli tirare di nuovo. Il giocatore è troppo legato ai dadi: sa quanti sono, come usarli e come calcolare il totale. Se qualcosa sui dadi cambia anche la classe "Giocatore" deve cambiare. Utilizziamo una **Pure Fabrication** per gestire i dati. Cup contiene i dadi e ha la responsabilità di tirarli e calcolare il totale. La cup supporta anche l'indirection dato che si mette tra il player e i dadi. Per poter gestire le diverse caselle ovviamente utilizziamo una classe polimorfa e sarà proprio il giocatore a far partire il metodo polimorfo "Sono finito su di te, esegui la tua azione!". Per far sì che la casella possa interagire con il giocatore quest'ultimo gli viene passato come parametro al metodo polimorfo landedOn(). Per far funzionare i comportamenti polimorfi è possibile aggiungere attributi e metodi alla classe contesto (giocatore). Un altro miglioramento è quello di dare al giocatore la sua posizione in modo tale da non passare ogni volta per il segnalino (per questo motivo il segnalino potrebbe non esser necessario nel modello software). L'implementazione della funzione landedOn è un'applicazione del design pattern **strategy**. Dove il **player** è l'**oggetto contesto** e i **comportamenti** specifici che avvengono quando il giocatore finisce su una casella sono le **strategie**. Quindi i diversi tipi di **casella** sono gli **oggetti strategia**.

3.2 Iterazione 3

Nell'iterazione tre appaiono delle nuove caselle, Lot (Proprietà), Railroad (Ferrovia) e Utility (Società). Un giocatore può quindi acquistarla se non è posseduta da nessuno, se è posseduta da se stesso non fa nulla, se è di proprietà di un'altro giocatore deve pagare un affitto. Queste tre diventeranno poi sottoclassi di una classe astratta **Property Square**. Questo perché il calcolo dell'affitto per ognuna di queste proprietà è differente, viene applicato quindi **Template Method** che viene utilizzato per gestire algoritmi che hanno la stessa struttura generale ma variano in alcuni passi. Funziona principalmente che nella superclasse Property Square landedOn è il metodo template dato che contiene passi comuni a tutte le proprietà. Per i passi che variano si definisce un metodo astratto getRent nella superclasse PropertySquare. Nelle sottoclassi avviene l'override di quel metodo applicando la sua logica di affitto. Differenze tra **strategy** e **Template method**, template usa l'ereditarietà per variare parti di un algoritmo fisso. **Strategy** usa la delega per variare l'intero algoritmo. Quindi template aiuta a gestire algoritmi simili con variazioni locali in modo pulito senza ripetizioni.